

Detecting Anomalous Communication Behaviors in Dynamically Evolving Networked Systems

Mehedi Hassan¹, Mehmet Engin Tozal², Vipin Swarup³, *Member, IEEE*, Steven Noel⁴, Raju Gottumukkala⁵, and Vijay Raghavan⁶, *Life Senior Member, IEEE*

Abstract—Devices in real-world networked systems typically exhibit a consistent communication behavior over time. Hence, anomalous communication behaviors may often indicate cyberattacks taking place in these systems. As a result, identifying anomalous communication links can help us to activate appropriate response mechanisms to ensure the integrity and security of such networked systems. In this study, we propose an approach to detect anomalous communication links in dynamically evolving networked systems. Furthermore, we employ the External Direct Sum of Vector Spaces (EDSoVS) to enable edge/dyad representation learning from node representations. We then use the dyad representations as features to train an SVM model and detect future anomalous links. We evaluate our approach on a synthetic dataset representing a real-world communication network provided by the MITRE Corporation and twelve real-world datasets provided by Cisco. Our empirical results show that the accuracy rates of the proposed model vary between 70.54% to 95.79%, while the F1 score is between 78.22% and 95.97%. We compare our approach against four techniques involving neural networks and graph neural networks. We show that our approach achieves higher accuracy, precision, and recall rates.

Index Terms—Anomalous link detection, spectral embedding, edge embedding, dynamic graphs, network security.

I. INTRODUCTION

A NETWORKED system consists of devices communicating through a shared medium to carry out a task and/or share resources [1]. The communication medium can be wired, wireless, or a combination of them. Besides, these networked systems may be in the form of Personal Area Networks (PAN), Home Area Networks (HAN), Local Area Networks (LAN), Metropolitan Area Networks (MAN), or Wide Area Networks (WAN). The hosts or devices within networked systems often exhibit consistent, but diverse roles. For example, computing devices handle information processing, sensors collect and forward data, client devices consume data and services, servers provide data and services, and peer devices assume both roles

[2]. Although hosts may undertake various roles, these roles stay consistent over time. Hence, networked systems demonstrate a stable communication behavior at consecutive time intervals [3], where the duration of the time interval depends on the application domain. On the other hand, these systems may display atypical or anomalous behaviors during cyberattacks. Such attacks may involve vulnerability scanning, unauthorized data access, or disruption of critical services. Anomalous communication examples may involve but are not limited to, (i) an internal assailant sporadically scanning computers to exploit known or unknown vulnerabilities; (ii) an employee from one sub-domain attempting to access computers in another sub-domain within the organization; and (iii) a server-resident worm trying to spread to previously serviced client computers. Detecting such anomalous communication behavior is critical for identifying and addressing potential security threats within networked systems. Our proposed approach can also be integrated into intrusion detection systems to detect unexpected communication behaviors at an early stage.

Real-world networked systems evolve over time [4]. As a result, dynamic graph representations are increasingly employed to model the evolving relationships in these networks. Recently, numerous studies explored anomaly detection in dynamic graphs [5], [6], [7], [8]. However, these studies only detect anomalous nodes or node behaviors in networked systems. In addition, these methods are typically developed for undirected graph representations. On the other hand, directed representations of dynamic graphs enable a clear and precise depiction of how data flows between devices.

In this study, we employ directed edge embeddings to identify anomalous communication links, rather than anomalous nodes. Effectively learning edge representations requires encoding node representations into a low-dimensional space. To that end, we initially leverage the spectral embedding that encodes the node representations of a graph into a low-dimensional embedding space [9]. Spectral embedding is applied to undirected graphs as the corresponding Laplacian matrix is symmetric and positive semi-definite. On the other hand, the Laplacian matrices of directed graphs are typically asymmetric. That is, the eigen-decomposition of the Laplacian leads to complex eigenvalues [10]. Chung proposed a random-walk strategy using transition probability matrices and associated stationary distributions to obtain a symmetric Laplacian matrix for directed graphs [11]. However, the method

Received 10 February 2024; revised 6 July 2024; accepted 4 February 2025. Date of publication 5 March 2025; date of current version 20 March 2025. This work was supported by MITRE's Independent Research and Development Program under Award MITRE-1161269. The associate editor coordinating the review of this article and approving it for publication was Prof. Kemal Akkaya. (*Corresponding author: Mehedi Hassan.*)

Mehedi Hassan, Mehmet Engin Tozal, and Vijay Raghavan are with the School of Computing and Informatics, University of Louisiana at Lafayette, Lafayette, LA 70504 USA (e-mail: mehedicse@iut-dhaka.edu).

Vipin Swarup and Steven Noel are with The MITRE Corporation, McLean, VA 22102 USA.

Raju Gottumukkala is with the Department of Mechanical Engineering, University of Louisiana at Lafayette, Lafayette, LA 70504 USA.

Digital Object Identifier 10.1109/TIFS.2025.3548544

needs to address the possibility of a random walker getting trapped in a sub-region of the graph [12].

To avoid this, we adopt a technique presented in [12] for applying spectral embedding to directed graphs. We divide the set of nodes in the graph representing a networked system into two subsets: *out*-nodes and *in*-nodes. Then, we add the directed edges between *out*-nodes and *in*-nodes. The transformed graph becomes twice its original size. The additional entries in the adjacency matrix of the transformed graph comprise diagonal of *in*-degree and *out*-degree matrices and the transpose of the weighted adjacency matrix of the directed graph [12]. The Laplacian matrix of the transformed graph becomes symmetric and positive semi-definite. Hence, the spectral embedding technique can be applied to generate node embeddings for the transformed directed graphs.

The vector difference operator is commonly used to represent an edge between two nodes [13]. However, it can often generate the same representation of two separate edges between two distinct nodes, making it difficult to distinguish the edge representations in the embedding space. To solve this issue, in this study, we propose employing the External Direct Sum of Vector Spaces (EDSoVS) [14] to enable edge/dyad representation learning from the node representations. EDSoVS effectively maps two separate edges between two distinct nodes to different representations in the embedding space by concatenating the node embedding vectors. We use these dyad embeddings as features to be fed into a Support Vector Machine (SVM) classifier to detect anomalous links.

We partner with the MITRE Corporation in this study. Hence, we evaluate our proposed model on a synthetic dataset representing a real-world communication network of interest to MITRE. In addition, we use twelve real-world datasets of different organizations shared by Cisco [15] to validate our results further. We experimented with undirected and directed graphs, their unweighted (semi-weighted for directed graphs) and weighted graph representations, as well as EDSoVS and difference vector representations of the edges. Despite the presence of data collection artifacts in some daily snapshots of five Cisco network datasets that negatively affect the empirical results, our proposed model with EDSoVS indicates that the average accuracy scores vary between 70.54% to 95.79%, and the average F1 scores vary between 78.22% to 95.97% across different datasets. If we omit the affected daily snapshots, the average accuracy scores of our proposed model vary between 86.12% to 97.93%, and the average F1 scores vary between 87.13% to 97.95% across different datasets. Please see Section IV-D2 for further details. We also compare our approach with two embedding techniques based on neural networks, *i.e.*, Node2Vec [13] and DeepWalk [16], and two current techniques based on graph neural networks, *i.e.*, DGCN [17] and TADDY [18]. We show that our approach consistently achieves better results.

The main contributions of this study are two-fold:

- We propose an approach to identify anomalous communication links in evolving networked systems.
- We propose employing the External Direct Sum of Vector Spaces (EDSoVS) to effectively represent edge/dyad embeddings in the embedding space.

The rest of the paper is organized as follows. Section II presents the related work. In Section III, we explain the process of generating node and edge embeddings. Section IV presents the descriptions of the datasets, experimental design, and empirical evaluations of our approach. In Section V, we present comparative analyses of our proposed approach relative to four techniques involving neural networks and graph neural networks. Finally, Section VI concludes the study.

II. RELATED WORK

Recent approaches to graph anomaly detection employ node representations or embeddings. Rahman et al. [19] propose a statistical approach for identifying anomalous node behavior in social networks. Gu et al. [20] encode graphs into low-dimensional vector representations to measure node centrality and detect communities. Feng et al. [21] introduce a novel network embedding technique incorporating a “degree penalty” principle. Liu et al. [22] improve the identification of novel and unfamiliar anomalous nodes using a Bayesian statistical model. CenGCN [23] captures the variance in information flow among vertices. These studies generate node embeddings to detect anomalous nodes or node behaviors using static graph representations. We focus on dynamically evolving graphs in this study.

Other studies use dynamic graphs for anomalous node detection. Tian et al. [7] propose an anomalous node detector method using Generative Adversarial Networks (GANs). Bars et al. [5] detect node-level anomalies using a probabilistic framework. Heard et al. [8] use discrete time counting process models to detect anomalous nodes. Ding et al. [6] detect anomalies in attributed networks using deep neural networks. Our focus in this study is to detect anomalous edges rather than anomalous nodes.

Some studies detect anomalous links in dynamic graphs. Cai et al. [24] detect sub-graph structural changes centered on the target edge in a given time window. Xu et al. [25] propose a general detection pipeline by sampling sub-graphs to detect adversarial attacks in graph neural networks (GNNs). King and Huang [26] use an intrusion detection framework to detect anomalous traffic involving network lateral movement. Lo et al. [27] detect anomalous traffic in IoT networks using graph neural networks (GNNs). However, they do not account for unobserved links. Bhatia et al. [28] propose an anomaly detection technique by detecting anomalous edge streams. Eswaran and Faloutsos [29] use a random walk-based edge anomaly scoring function to detect anomalous edge streams. These studies sample edge-based sub-graphs to detect anomalies or use random walk-based techniques to detect anomalous edge streams. We focus on anomalous link detection in this study instead of detecting anomalous sub-graphs or edge streams.

Fewer studies use neural networks over directed graphs for node classification. Zhang et al. [30] propose a magnetic Laplacian-based neural network over directed graphs for node classification. Tong et al. [31] use a spectral-based neural network over directed graphs for node classification. Gasteiger et al. [32] introduce directional message passing to leverage

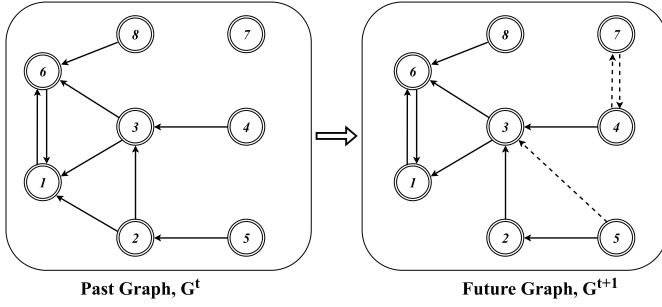


Fig. 1. An example of edges appearing and disappearing in a networked system between two time intervals t and $t + 1$.

direction information to predict molecular properties. However, these studies do not detect anomalous links.

III. SPECTRAL EMBEDDINGS

In this section, we explain the process of generating node embeddings for undirected and directed graphs using Laplacian eigenmap. Then, we introduce a procedure to generate edge embeddings based on embeddings of nodes in a graph.

A. Problem Definition

Real-world networked systems dynamically evolve over time. Let G denote a dynamic graph that represents an evolving networked system. Given that the current time is t ; the time interval between $t - 1$ and t indicates the “past”, and the time interval between t and $t + 1$ indicates the “future”. The “past graph”, $G^t = (V^t, E^t)$, represents a snapshot of G between time interval $t - 1$ and t . The set of nodes or vertices, V^t , corresponds to the devices, and the set of links or edges, E^t , represents the communication taking place between pairs of nodes in the networked system between time $t - 1$ and t . Figure 1 illustrates a dynamically evolving, networked system. In the figure, the solid edges represent repeated connections, and dashed edges represent new connections forming between two consecutive time intervals. Also, the connection between node pair (2,1) disappears from time t to $t + 1$.

Based on historical data, anomalous or unexpected links are communication attempts that demonstrate unusual or unexpected behavior, while non-anomalous or expected links adhere to predictable patterns within the networked system. Given a past graph $G^t = (V^t, E^t)$, our goal is to detect anomalous links occurring in the future graph, $G^{t+1} = (V^{t+1}, E^{t+1})$. Note that the addition of new nodes into the system requires the approval of a network administrator. Therefore, we assume that the universal set of nodes are known in advance. However, we do not necessitate that every node participates in a communication at all time intervals. On the other hand, new links may appear, and old links may disappear at different time intervals.

B. Node Embeddings for Undirected Graphs

Let $G^t = (V^t, E^t)$ be a weighted, undirected graph. The weighted adjacency matrix, $W^t \in \mathbb{R}^{|V| \times |V|}$, is defined as:

$$W^t = \begin{cases} w_{ij}^t > 0 & \text{if } v_i^t \text{ and } v_j^t \text{ are connected, and } i \neq j \\ 0 & \text{otherwise} \end{cases} \quad (1)$$

where $|V|$ is the number of nodes in the universal set, and w_{ij}^t is the weight of the link between v_i^t and v_j^t . Note that when G^t is an unweighted graph, existing links between node pairs (v_i^t, v_j^t) have a weight of 1. The degree matrix, $D^t \in \mathbb{R}^{|V| \times |V|}$, is a diagonal matrix consisting of the degree of the nodes in G^t . The diagonal elements of D^t are defined as:

$$d_i^t = \sum_{j=1}^{|V|} w_{ij}^t \quad (2)$$

where d_i^t is the degree of node v_i^t . The weighted adjacency matrix (W^t) and degree matrix (D^t) are square matrices with dimension $|V| \times |V|$.

The Laplacian matrix is a representation of a graph derived from its adjacency and degree matrices. The Laplacian matrix of G^t is defined as:

$$L^t = D^t - W^t \quad (3)$$

In Equation 3, nodes with a high degree contribute to a large diagonal entry in the Laplacian matrix. Therefore, the Laplacian matrix is normalized to enhance its robustness against the nodes with higher degrees. The symmetrically normalized Laplacian matrix is defined as:

$$L_{sym}^t = D^{-1/2} W^t D^{-1/2} \quad (4)$$

Laplacian eigenmap is a non-linear dimensionality reduction method widely employed in spectral clustering [33]. It involves computing the eigenvectors of a Laplacian matrix, enabling the representation of graph data in a lower-dimensional space. The approach is particularly effective in preserving the local structure of the data [33]. Laplacian eigenmap ensures that nodes close to one another in the original graph remain similarly situated in the reduced space.

Our objective is to extract p -dimensional node embeddings represented by, $Y^t = [(y_1^t)^T, (y_2^t)^T, \dots, (y_{|V|}^t)^T] \in \mathbb{R}^{|V| \times p}$, where $p \leq |V|$ (usually $p \ll |V|$), and $\top$ is the transpose operator. The Laplacian eigenmap is expressed as an optimization problem in Equation 5.

$$\underset{Y^t}{\text{minimize}} \sum_{i=1}^{|V|} \sum_{j=1}^{|V|} w_{ij}^t \|y_i^t - y_j^t\|_2^2 \quad (5)$$

Equation 5 implies that if v_i^t and v_j^t are close to each other, then their corresponding weight w_{ij}^t will be large. As y_i^t and y_j^t are the embeddings of v_i^t and v_j^t , minimizing $\|y_i^t - y_j^t\|_2^2$ makes two nodes closer to each other in the embedding space. Therefore, the Laplacian eigenmap is suitable for local data fitting as it prioritizes nearby points and aims to preserve the global structure through this local fitting approach.

The optimization problem in Equation 5 can also be expressed as follows:

$$\underset{Y^t}{\text{minimize}} \quad \text{tr}(Y^{t\top} L_{sym}^t Y^t) \quad (6a)$$

$$\text{subject to} \quad Y^{t\top} Y^t = I \quad (6b)$$

The Lagrangian for this optimization problem [34] is:

$$\mathcal{L}^t = Y^{t\top} L_{sym}^t Y^t - \text{tr}(\Lambda^{t\top} (Y^{t\top} Y^t - I)) \quad (7)$$

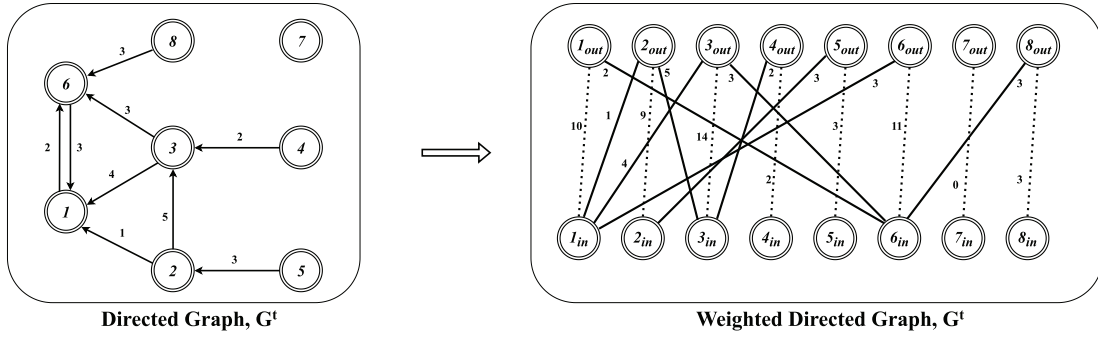


Fig. 2. A weighted, directed graph transformation example.

where Λ^t is the diagonal matrix with dual variables. Setting the derivative of Lagrangian to zero gives us the eigenvalue problem for the Laplacian matrix, defined as:

$$\mathbb{R}^{|V| \times |V|} \ni \frac{\partial \mathcal{L}^t}{\partial Y^t} = 2L_{sym}^t Y^t - 2Y^t \Lambda^t \stackrel{\text{set}}{=} 0 \Rightarrow L_{sym}^t Y^t = Y^t \Lambda^t \quad (8)$$

where columns of Y^t are the eigenvectors. The columns of matrix Y^t are arranged in ascending order based on their corresponding eigenvalues. The eigenvector with a zero eigenvalue is discarded as it indicates the number of connected components of G^t . We select the eigenvectors with p smallest non-zero eigenvalues from the sorted Y^t to obtain an embedding space with p -dimensions. Consequently, Y^t is transformed to an $|V| \times p$ matrix, where each row represents a p -dimensional embedding vectors of the nodes in an undirected graph.

C. Node Embeddings for Directed Graphs

The direction information in directed graphs enables the differentiation between a link from one node to another and its reverse. However, the Laplacian matrix of a directed graph is typically asymmetric and not positive semi-definite. That is, the eigen-decomposition of the Laplacian yields complex eigenvalues. Therefore, the resulting node embeddings cannot naturally be represented in a real embedding space.

To address this issue, we transform the directed graph to obtain a symmetric Laplacian matrix as proposed in [12]. We partition the original set of nodes, V^t , into two subsets, i.e., *out*-nodes (V_{out}^t) and *in*-nodes (V_{in}^t). We add directed edges between V_{out}^t and V_{in}^t , and assign their edge weight according to the original directed graph. We also insert an artificial edge between *in* and *out* versions of every node in V^t and assign a weight equivalent to the sum of the *in*-degree and *out*-degree of that specific node. Figure 2 demonstrates an example of the modification of a weighted, directed graph. In the figure, solid edges represent the edges from the original directed graph, and dashed edges represent the artificially added edges between *in* and *out* versions of each node. A directed edge between node pairs (v_i^t, v_j^t) is replaced with an edge between node pairs $(v_{i,out}^t, v_{j,in}^t)$, maintaining the direction of the original edge. If the directed graph is unweighted, all solid edges have a weight 1 and dashed edges (artificially added edges) have a weight equivalent to the sum of degrees of each node. As a result, the original unweighted, directed graph is transformed into a semi-weighted, undirected graph.

The weighted adjacency matrix of the transformed graph, W^t , is defined as:

$$W^t = \begin{bmatrix} 0 & W_d^t + D_{in}^t + D_{out}^t \\ W_d^{t^T} + D_{in}^t + D_{out}^t & 0 \end{bmatrix} \quad (9)$$

where W_d^t is the weighted adjacency matrix, D_{in}^t is the *in*-degrees, and D_{out}^t is the *out*-degrees of the original directed graph. Let D^t be the diagonal degree matrix of W^t with the degrees $d_{1,out}^t, \dots, d_{|V|,out}^t, d_{1,in}^t, \dots, d_{|V|,in}^t$, where $d_{i,out}^t = d_{i,in}^t + 2 \times d_{i,out}^t$ and $d_{i,in}^t = 2 \times d_{i,in}^t + d_{i,out}^t$. The transformed graph has $2|V|$ nodes, and the dimension of its weighted adjacency and degree matrix is $2|V| \times 2|V|$. The Laplacian is now symmetric and positive semi-definite. The symmetrically normalized Laplacian matrix can be computed using Equation 4 with the updated W^t and D^t . The Laplacian eigenmap can now be applied to the transformed graph to compute node embeddings.

D. Edge Embeddings

A natural way to represent an edge between the vector embeddings of two nodes is to use the vector difference operator. However, the vector difference operator may map two separate edges between two distinct nodes to the same representation. To illustrate this, Figure 3a shows a sample graph consisting of four nodes and three edges. The example embeddings of the nodes are shown in Figure 3b, and the vector differences of two edges are highlighted in the same figure. As clearly shown in Figure 3c, the edge representations of $e_{1,2}$ and $e_{3,4}$ map two distinct edges to the same difference vector. This problem can be observed in similar representations such as vector addition, dot product, or Hadamard product for vectors [13].

To overcome this issue, we propose using the External Direct Sum of Vector Spaces (EDSoVS) [14] to represent the embeddings of edges. The EDSoVS is a method of combining vector spaces to form a new vector space [14]. It practically concatenates the embeddings of the nodes incident on an edge in the new vector space. The EDSoVS of Y^t with itself, denoted as $Y^t \oplus_e Y^t$, where $y^t \in Y^t$, consists of all ordered pairs. The vector space operations on $Y^t \oplus_e Y^t$ are defined component-wise:

$$(y_i^t, y_k^t) + (y_j^t, y_l^t) = (y_i^t + y_j^t, y_k^t + y_l^t) \quad (10)$$

and

$$c \cdot (y_i^t, y_k^t) = (c \cdot y_i^t, c \cdot y_k^t) \quad (11)$$

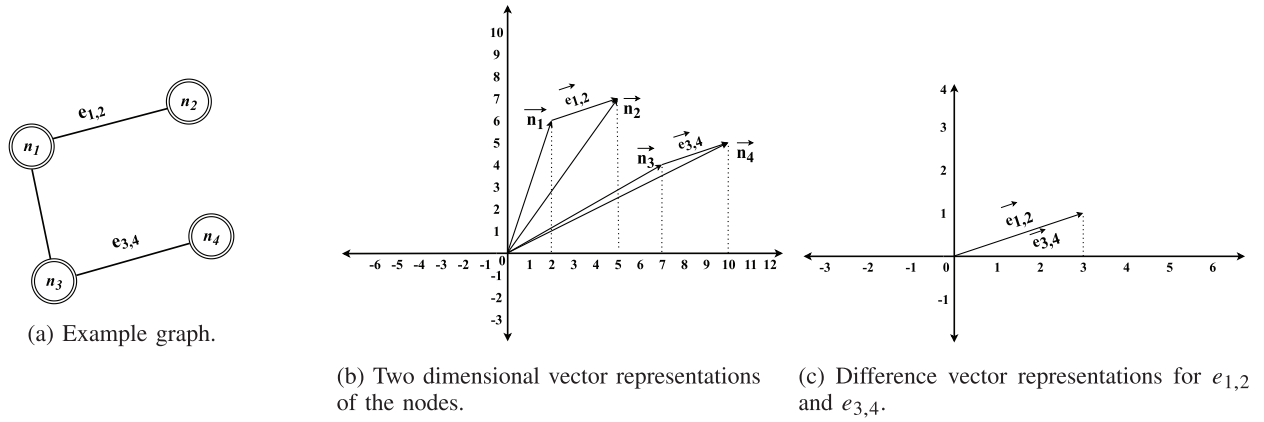


Fig. 3. Illustration of edge embeddings using the difference vector representations of edges.

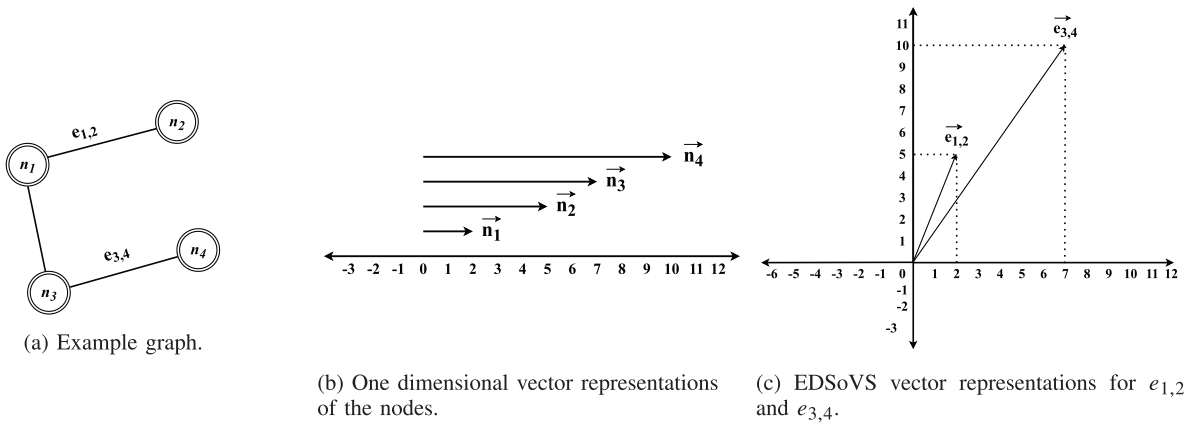


Fig. 4. Illustration of edge embeddings using the EDSoVS vector representations of edges.

The dimension of $Y' \oplus_e Y'$ is twice the dimension of Y' . Figure 4 demonstrates an example of EDSoVS vector representations of the same edges using the same graph in Figure 3. As EDSoVS increases the dimension by two folds, using two-dimensional vectors for node embeddings yields four-dimensional vectors, which is not possible to draw. Hence, we use one-dimensional vectors for node embeddings as shown in Figure 4b. EDSoVS vector representations generate two-dimensional vectors for edges. In Figure 4c, the edge representations of $e_{1,2}$ and $e_{3,4}$ map two distinct edges to two different vectors, making them distinguishable in the embedding space. Unlike the vector difference or similar representations, the EDSoVS vector representations of edges can differentiate between two distinct edges in the embedding space.

IV. EXPERIMENTAL RESULTS

In this section, we first present the datasets used in this study and the preprocessing steps in depth. Next, we introduce our experimental design. Then, we discuss the experimental results of the proposed anomalous link detection approach. Finally, we present comparative analyses in Section V.

A. Dataset Description

We conducted this study with our partners from the MITRE Corporation. Hence, the primary dataset we used in this

study is a synthetically generated dataset provided by MITRE. They have introduced a probabilistic model for generating a diverse set of graphs that simulate a reference graph containing authentic network flows. The model is designed to create dynamic multigraphs with edge attributes such as port numbers and protocol types. The modeling process consists of three essential stages for scalability and efficiency: *structure generation*, *feature generation*, and *graph alignment*. In the structure generation stage, a stochastic Kronecker graph generator [35] transforms the reference dynamic multigraph into a static simple graph. The feature generation stage uses a CTGAN model [36] to learn the distribution of the edge features. Finally, the graph alignment stage utilizes an XGBoost model [37] to overlay the features onto the graph and determine the source and destination of each edge. The segmentation of these stages enables straightforward generalization to graphs with varying numbers of nodes and edges compared to the reference graph, in addition to achieving acceptable similarity and diversity scores to ensure that the synthetic graphs are similar to the reference graphs while being diverse to conceal the details of the original graphs. The dataset contains an eight-day network flow information, including source and destination IP addresses, start and finish timestamps, destination port, and protocol.

In order to validate our results further, we also use twelve real-world datasets shared by Cisco in 2022 [15]. These

datasets were collected from different organizations using software sensors developed by Cisco Secure Workload. Although Cisco shares 22 distinct datasets, we use twelve datasets that have less than 10,000 nodes. In fact, this limitation is imposed by the eigen-decomposition problem, which has $\mathcal{O}(|V|^2)$ memory complexity and $\mathcal{O}(|V|^3)$ computational complexity “in practice” for an $|V| \times |V|$ matrix [38]. The dataset is gathered over a span of four days and includes anonymized IP addresses, port numbers, and an edge list between the IP addresses.

B. Dataset Preprocessing

Based on the starting timestamps, we divided the MITRE dataset into eight individual days. Day₈ data consists of only four edges and eight nodes, appearing just after midnight of Day₇. Therefore, we discard Day₈ data and use the data from the initial seven days. Cisco network datasets span over four days, and the data for each day is provided in a separate text file. We present the day-wise statistics of all 13 datasets in Table I. The number of edges denotes multiple connections, and the number of unique edges denotes distinct connections taking place between two devices in a 24-hour time period. In addition, we discovered 467 to 501 nodes in the MITRE dataset and 14 to 5168 nodes in different Cisco network datasets that appear or disappear over time. Such nodes are part of their respective networks, hence, we add these nodes as isolated nodes to their daily graphs. As a result, nodes appearing over the entire duration of the network graph data are included in each graph snapshot. Note that in Table I, the number of nodes on Day₁ of Cisco G-8, G-13, G-17, and G-18 datasets, as well as Day₁ and Day₄ of Cisco G-20 dataset exhibit an irregular pattern compared to the other days of the same datasets. In fact, this irregularity is attributed to malfunctioning devices, deploying sensors only to a subset of the network, and switching off sensors to avoid consuming too much computational resources [15]. Although they affect our results adversely, we decided to include these datasets in our experiments for the sake of completeness.

We preprocess the daily network datasets to obtain the training and test datasets. We convert the daily graphs into undirected and directed graphs. We create weighted representations of the undirected and directed graphs, where edge weights correspond to the number of multiple edges. Then, we create unweighted representations of the undirected and semi-weighted versions of the directed graphs, where edge weights are set to 1. We first obtain the edge lists based on the observed or existing links between node pairs from the graphs. We incorporate two types of non-existing links between node pairs in our datasets, namely, two-way-missing links and one-way-missing links. A non-existing link in both directions is considered as a “two-way-missing link”, while a non-existing link in the opposite direction of an existing link is considered as a “one-way-missing link”. The numbers of existing and non-existing links between node pairs are significantly imbalanced. To address this issue, we use random sampling on the non-existing links between node pairs and form balanced datasets. Finally, the dataset generated from Day_{*t*} of a graph is used as the training dataset, while the dataset generated from Day_{*t+1*}

TABLE I
DAY-WISE STATISTICS OF THE MITRE AND CISCO NETWORK DATASETS

Dataset	Days	No. of Nodes	No. of Isolated Nodes	No. of Total Nodes	No. of Edges	No. of Unique Edges	Density
MITRE	Day ₁	1335	487	1822	32197	1810	0.00055
	Day ₂	1328	494	1822	31738	1811	0.00055
	Day ₃	1330	492	1822	29614	1817	0.00055
	Day ₄	1355	467	1822	28265	1818	0.00055
	Day ₅	1321	501	1822	31618	1815	0.00055
	Day ₆	1346	476	1822	29102	1816	0.00055
	Day ₇	1326	496	1822	30038	1811	0.00055
Cisco G-1	Day ₁	1063	384	1447	537841	96947	0.04633
	Day ₂	1075	372	1447	633449	98563	0.04711
	Day ₃	1074	373	1447	607584	97591	0.04664
	Day ₄	1083	364	1447	620785	107841	0.05154
Cisco G-7	Day ₁	237	53	290	2861	559	0.00667
	Day ₂	236	54	290	3176	532	0.00635
	Day ₃	251	39	290	3180	631	0.00753
	Day ₄	228	62	290	3179	529	0.00631
Cisco G-8	Day ₁	2182	1207	3389	31629	6424	0.00056
	Day ₂	2815	574	3389	61694	10331	0.00090
	Day ₃	2710	679	3389	58896	10427	0.00091
	Day ₄	2805	584	3389	61325	10357	0.00090
Cisco G-11	Day ₁	174	33	207	6760	1345	0.03154
	Day ₂	172	35	207	7605	1247	0.02924
	Day ₃	170	37	207	7619	1194	0.02800
	Day ₄	168	39	207	7567	1253	0.02938
Cisco G-12	Day ₁	1527	927	2454	11632	2948	0.00049
	Day ₂	1494	960	2454	15188	2997	0.00050
	Day ₃	1427	1027	2454	14179	2861	0.00048
	Day ₄	1364	1090	2454	14452	2704	0.00045
Cisco G-13	Day ₁	122	5168	5290	1001	193	0.00001
	Day ₂	4513	777	5290	92694	20196	0.00072
	Day ₃	3845	1445	5290	85381	18181	0.00065
	Day ₄	4294	996	5290	88809	18618	0.00067
Cisco G-14	Day ₁	432	143	575	2960	648	0.00196
	Day ₂	425	150	575	3242	630	0.00191
	Day ₃	430	145	575	3223	644	0.00195
	Day ₄	431	144	575	3217	624	0.00189
Cisco G-16	Day ₁	170	68	238	1091	217	0.00385
	Day ₂	168	70	238	1186	215	0.00381
	Day ₃	170	68	238	1219	228	0.00404
	Day ₄	165	73	238	1166	211	0.00374
Cisco G-17	Day ₁	45	551	596	432	90	0.00025
	Day ₂	510	86	596	4853	849	0.00239
	Day ₃	466	130	596	4709	785	0.00221
	Day ₄	508	88	596	4886	853	0.00241
Cisco G-18	Day ₁	218	71	289	1269	213	0.00256
	Day ₂	261	28	289	2943	459	0.00551
	Day ₃	275	14	289	2952	445	0.00535
	Day ₄	257	32	289	2887	431	0.00518
Cisco G-19	Day ₁	68	18	86	431	114	0.01560
	Day ₂	50	36	86	446	85	0.01163
	Day ₃	53	33	86	450	85	0.01163
	Day ₄	59	27	86	470	97	0.01327
Cisco G-20	Day ₁	28	1459	1487	57	27	0.00001
	Day ₂	262	1225	1487	5552	955	0.00043
	Day ₃	264	1223	1487	5420	939	0.00042
	Day ₄	1424	63	1487	7545	2130	0.00096

of the same graph is used as the test dataset in our experiments. For the MITRE dataset, $t = \{1, 2, 3, 4, 5, 6\}$ and for Cisco network datasets, $t = \{1, 2, 3\}$.

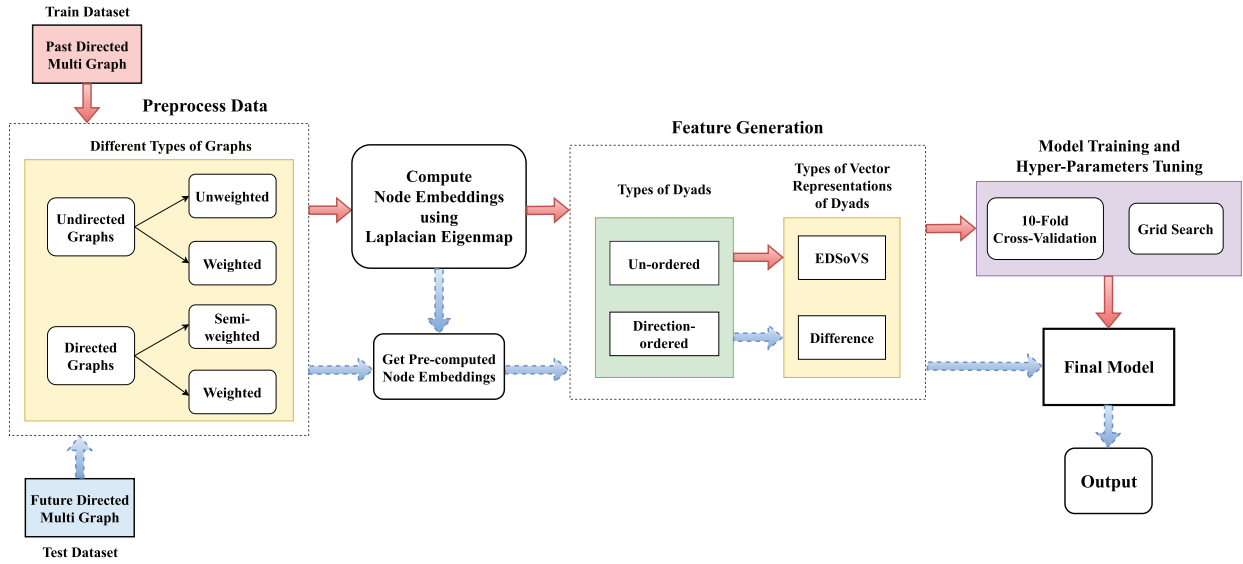


Fig. 5. Experimental design of our proposed approach.

C. Experimental Design

Figure 5 depicts the experimental design using undirected and directed graph representations. The red-solid arrows represent the flow of tasks related to training our model using a training dataset, while the black-dashed arrows represent the flow of tasks related to testing the model using the corresponding test dataset. It is important to note that we execute the training process before executing the tasks related to the test process. For each graph, we first preprocess the graph data to create the training and test datasets as described in Section IV-B. Then, we apply the Laplacian eigenmap to obtain node embeddings using the training dataset. We compute node embeddings for embedding sizes, $p = \{2, 4, 8, 16, 32, 64, 128\}$. The dimension of node embeddings matrix is $|V| \times p$ for undirected graphs and $2|V| \times p$ for directed graphs, where $|V|$ is the number of nodes and p is the node embedding size.

A *dyad* is just a pair of nodes which may or may not have a link in between. The concept of dyads allows us to extend the concept of edges to node pairs and develop representations for pairs of nodes with and without links between them. Dyads are un-ordered pairs of nodes in undirected graphs and direction-ordered pairs of nodes in directed graphs. Specifically, a dyad $\delta_{i,j}$ between node pair v_i and v_j is equivalent to dyad $\delta_{j,i}$ in undirected graphs, while $\delta_{i,j}$ and $\delta_{j,i}$ between the same node pair are not equivalent to each other in directed graphs. In our experimental design, dyads with links are considered to be expected, and dyads without links are considered to be anomalous.

We employ two types of vector representations to obtain embeddings for *dyads*: EDSoVS and vector difference. We use the pre-computed node embeddings from Day_t training datasets to generate embeddings for *dyads* in Day_{t+1} test datasets. These embeddings are used as features to train a Support Vector Machine (SVM) [39] as a binary classifier. SVM establishes a decision boundary (hyperplane) between two classes to maximize the margin of separation. It employs

a regularization parameter (C) to balance the misclassification penalty, thereby influencing the trade-off between bias and variance. Furthermore, SVM can handle nonlinear classification using the kernel trick, which involves mapping observations into a higher-dimensional feature space. Interested readers are referred to [39] for further details. Lastly, we conduct a 10-fold cross-validation to determine the optimal embedding size for the final model.

Training SVM is computationally costly. The time complexity for training SVM ranges from $\mathcal{O}((2|E|)^2 * 2p)$ to $\mathcal{O}((2|E|)^3 * 2p)$, where $|E|$ is the number of dyads in a graph snapshot, and $2p$ is the dimension of the dyad embeddings. However, we train the SVM classifier offline during the training stage. The dyad embeddings are computed from the pre-computed node embeddings. Nevertheless, one can use any other binary classifier.

D. Evaluations

In this part, we present and explain the empirical results based on unweighted, undirected, and semi-weighted, directed graph representations, using EDSoVS vector representation of *dyads*. We utilize the MITRE communication network dataset as our primary dataset. That is, we present detailed results for the MITRE dataset and briefer results for the Cisco network datasets due to the page limitations. Yet, the detailed results of the Cisco network datasets are available in the Appendix.

In the following, we first experiment with different embedding sizes on the training (Day_1) dataset to determine the optimal embedding size. Then, we classify the links from the test (Day_2) dataset as *anomalous* or *expected* and explain the classification results. We repeat this process for the remaining daily datasets using the same embedding size. That is, we train our model on the training (Day_t) datasets and classify the links on the test (Day_{t+1}) datasets. We present the day-wise classification results and summarize the performance

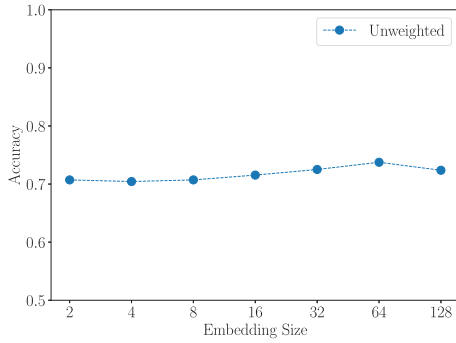


Fig. 6. Accuracy scores on the MITRE-Day₁ training dataset for different embedding sizes using the unweighted representation of undirected graphs and EDSoVS.

TABLE II

EVALUATION METRICS ON THE MITRE-DAY₂ TEST DATASET USING THE UNWEIGHTED REPRESENTATION OF UNDIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 64

Accuracy	Precision	Recall	F1 Score
72.75%	86.88%	53.58%	62.28%

TABLE III

CONFUSION MATRIX OF THE MITRE-DAY₂ TEST DATASET USING THE UNWEIGHTED REPRESENTATION OF UNDIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 64

Actual	Predicted	
	Expected	Anomalous
Expected	3306	316
Anomalous	1670	1952

of our model across all days. Finally, we replicate the same process for the Cisco network datasets and present their results. In the following, we present and discuss the empirical results of undirected and directed graphs separately.

1) *Undirected Graphs*: Figure 6 depicts the accuracy scores for different embedding sizes on the MITRE-Day₁ training dataset. The accuracy scores suggest that the optimal embedding size is 64. We set the hyperparameters of the SVM, regularization (C), gamma (γ), and kernel function to 100, 'scale', and RBF for the final model, respectively. We present the evaluation metrics on the MITRE-Day₂ test dataset using the embedding size 64 in Table II. In the table, the accuracy score is 72.75%, the precision score is 86.88%, the recall score is 53.58%, and the F1 score is 62.28%. We present the confusion matrix in Table III to analyze the results further. In the table, our proposed model misclassifies 316 out of 3622 (8.73%) expected dyads and 1670 out of 3622 (46.11%) anomalous dyads. Due to the unavailability of the direction information, our model misclassifies one-way missing anomalous links as expected links. Figure 7 presents a simple visualization of the expected and anomalous links using the first eigenvector of node embeddings for the undirected graph. The figure demonstrates that most of the anomalous links overlap with expected links.

We present the day-wise evaluation metrics in Table IV. As shown in the table, the model maintains consistent

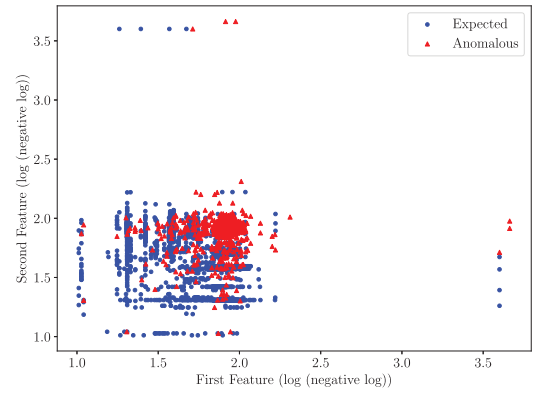


Fig. 7. Dyad feature visualization of the unweighted representation of undirected graph using the first eigenvector of node embeddings on the MITRE-Day₂ test dataset.

TABLE IV

PERFORMANCE SUMMARY OF MITRE DATASET USING THE UNWEIGHTED REPRESENTATION OF UNDIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 64

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day ₂	72.75%	86.88%	53.58%	66.28%
Day ₃	72.48%	86.24%	53.48%	66.02%
Day ₄	72.11%	82.92%	55.69%	66.63%
Day ₅	71.70%	82.05%	55.55%	66.25%
Day ₆	71.78%	82.93%	54.85%	66.03%
Day ₇	72.37%	84.73%	54.57%	66.39%
Average	72.20%	84.29%	54.62%	66.27%

TABLE V

PERFORMANCE SUMMARY OF CISCO NETWORK DATASETS USING THE UNWEIGHTED REPRESENTATION OF UNDIRECTED GRAPHS AND EDSOVS. ALL SCORES REPORTED ARE AVERAGED ACROSS ALL DAYS FOR THEIR RESPECTIVE DATASETS

Dataset	Embedding Size	Accuracy	Precision	Recall	F1 Score
Cisco G-1	64	79.49%	78.27%	73.14%	76.57%
Cisco G-7	16	88.83%	86.89%	92.23%	89.34%
Cisco G-8	16	88.33%	85.25%	96.09%	89.74%
Cisco G-11	64	78.97%	81.01%	76.24%	78.34%
Cisco G-12	16	85.55%	83.06%	90.80%	86.42%
Cisco G-13	8	76.12%	75.40%	91.24%	80.92%
Cisco G-14	32	83.00%	80.11%	88.00%	83.84%
Cisco G-16	16	91.36%	88.50%	95.60%	91.79%
Cisco G-17	16	78.92%	76.65%	95.95%	83.94%
Cisco G-18	16	89.65%	91.57%	87.05%	88.89%
Cisco G-19	32	78.69%	78.68%	80.83%	78.29%
Cisco G-20	32	69.59%	68.71%	87.82%	75.55%

performance across all daily MITRE datasets, where the difference is between 2 to 4 points. The model achieves an accuracy score of 72.20%, precision score of 84.29%, recall score of 54.62%, and F1 score of 66.27% on average across all daily test datasets.

Finally, we repeat the entire process for the Cisco network datasets using the same hyper-parameters and summarize our evaluation metrics in Table V. In the table, average accuracy scores vary between 69.59% to 91.36%, average precision scores vary between 68.71% to 91.57%, average recall scores

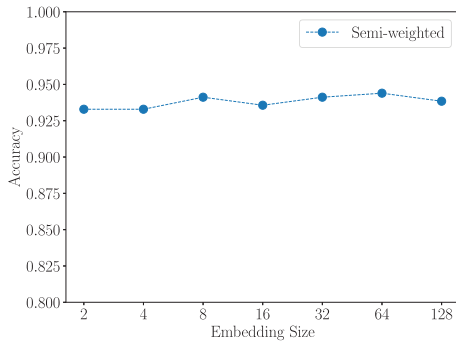


Fig. 8. Accuracy scores on the MITRE-Day₁ training dataset for different embedding sizes using the semi-weighted representation of directed graphs and EDSovS.

TABLE VI

EVALUATION METRICS ON THE MITRE-DAY₂ TEST DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSovS AND BEST EMBEDDING SIZE 64

Accuracy	Precision	Recall	F1 Score
94.15%	90.85%	98.18%	94.37%

TABLE VII

CONFUSION MATRIX OF THE MITRE-DAY₂ TEST DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSovS AND BEST EMBEDDING SIZE 64

Actual	Predicted	
	Expected	Anomalous
Expected	1692	119
Anomalous	33	1778

vary between 73.14% to 96.09%, and average F1 scores vary between 75.55% to 91.79%.

2) *Directed Graphs*: Figure 8 depicts the accuracy scores for different embedding sizes using the MITRE-Day₁ training dataset. The accuracy scores suggest that the optimal embedding size is also 64 for the directed graphs. We set the hyperparameters of the SVM, regularization (C), gamma (γ), and kernel to 100, 'scale', and RBF for the final model, respectively. Table VI presents the evaluation metrics on the MITRE-Day₂ test dataset using the embedding size 64. In the table, the accuracy score is 94.15%, the precision score is 90.85%, the recall score is 98.18%, and the F1 score is 94.37%. We also present the associated confusion matrix in Table VII. According to Table VII, the model adeptly identifies both expected and anomalous links. Our proposed model misclassifies 119 out of 1811 (6.57%) expected dyads and 33 out of 1811 (2.04%) anomalous dyads. Comparing Tables III and VII, we observe that the misclassification rate of the proposed model is reduced by 2 points for expected dyads and 44 points for anomalous dyads. Figure 9 presents a visualization of the expected and anomalous links using the first eigenvector of node embeddings for the directed graph. The figure demonstrates that most of the anomalous links form a cluster and are separated much better from the expected links.

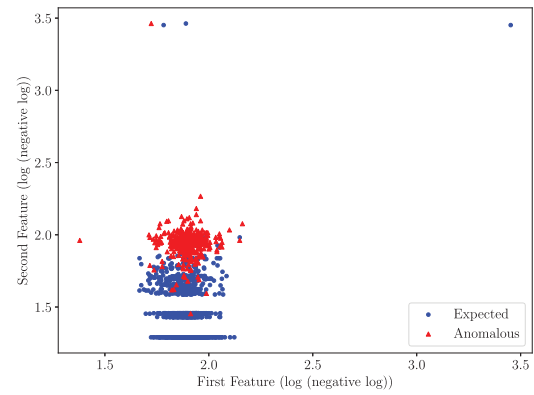


Fig. 9. Dyad feature visualization of the semi-weighted representation of directed graph using the first eigenvector of node embeddings on the MITRE-Day₂ test dataset.

TABLE VIII

PERFORMANCE SUMMARY OF MITRE DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSovS AND BEST EMBEDDING SIZE 64

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day ₂	94.15%	90.85%	98.18%	94.37%
Day ₃	93.23%	89.05%	98.57%	93.57%
Day ₄	95.35%	92.65%	98.51%	95.49%
Day ₅	94.87%	94.29%	95.53%	94.91%
Day ₆	94.66%	91.80%	98.07%	94.83%
Day ₇	92.96%	89.13%	97.85%	93.28%
Average	94.20%	91.30%	97.79%	94.41%

TABLE IX

PERFORMANCE SUMMARY OF CISCO NETWORK DATASETS USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS AND EDSovS. ALL SCORES REPORTED ARE AVERAGED ACROSS ALL DAYS FOR THEIR RESPECTIVE DATASETS

Dataset	Embedding Size	Accuracy	Precision	Recall	F1 Score
Cisco G-1	64	94.67%	92.62%	97.20%	94.83%
Cisco G-7	8	92.81%	89.08%	98.14%	93.28%
Cisco G-8	16	90.07%	86.74%	99.04%	91.79%
Cisco G-11	32	95.79%	93.53%	98.65%	95.97%
Cisco G-12	16	86.93%	80.57%	98.67%	88.52%
Cisco G-13	128	77.20%	74.72%	94.87%	82.26%
Cisco G-14	16	86.12%	82.29%	92.72%	87.15%
Cisco G-16	8	91.43%	87.57%	97.66%	92.14%
Cisco G-17	4	79.32%	76.35%	97.29%	84.29%
Cisco G-18	4	90.04%	86.14%	99.40%	91.68%
Cisco G-19	8	86.63%	84.02%	90.58%	87.13%
Cisco G-20	4	70.54%	68.38%	95.98%	78.22%

Table VIII presents the day-wise evaluation metrics. As shown in the table, the model maintains consistent performance across all daily MITRE datasets, where the difference is between 1 to 2 points. The model achieves an accuracy score of 94.20%, precision score of 91.30%, recall score of 97.79%, and F1 score of 94.41% on average across all daily test datasets.

Finally, we repeat the entire process for the Cisco network datasets using the same hyper-parameters and summarize our evaluation metrics in Table IX. In the table, average accuracy scores vary between 70.54% to 95.79%, average precision

scores vary between 68.38% to 93.53%, average recall scores vary between 89.38% to 99.40%, and average F1 scores vary between 78.22% to 95.97%. The large variance in the evaluation metrics is due to an inconsistent pattern observed in some graph snapshots within five Cisco network datasets. As we noted earlier in Table I, the number of nodes on Day₁ of Cisco G-8, G-13, G-17, and G-18 datasets, along with Day₁ and Day₄ of Cisco G-20 dataset, vary significantly compared to the other days of the respective datasets. For example, Day₁ of the Cisco G-13 dataset contains only 122 nodes, while Day₂, Day₃, and Day₄ contain 4513, 3895, and 4294 nodes, respectively. Similarly, Day₁ of the Cisco G-20 dataset contains 28 nodes, while Day₄ has 1424 nodes. In contrast, Day₂ and Day₃ have 262 and 264 nodes, respectively. Since the number of nodes varies significantly between consecutive days, a great number of node embeddings are not well represented. Therefore, our proposed model cannot generalize the prediction performance on the affected daily snapshots of these five datasets, which artificially decreases our performance across all days, in all metrics. If we discard the affected daily snapshots from our empirical results, the average accuracy scores of our proposed model vary between 86.12% to 97.93%, average precision scores vary between 80.57% to 97.93%, average recall scores vary between 90.58% to 99.40%, and average F1 scores vary between 87.13% to 97.95%.

3) *Worst-Case Scenario*: An intriguing scenario arises when a link appears precisely in the opposite direction of an expected link. For instance, consider a client typically initiating a connection to a database server based on past data. However, if the server initiates a connection to the client, an event that has never happened before, it is deemed unusual and regarded as a one-way missing anomalous link. To measure our performance on such links, we develop a separate test dataset only consisting of one-way missing links, referring to it as the worst-case dataset.

The model using undirected graphs cannot predict the direction of the links. In contrast, the model using directed graphs leverages the directional information. Hence, it can detect one-way missing links. We evaluate the trained models on the worst-case datasets to support our claim. Table X summarizes the performance of the proposed model on the *worst-case datasets*. In the table, accuracy scores vary between 71.55% to 99.36%. Our proposed approach can accurately detect one-way missing links. Note that the precision score in Table X is 100%, and recall exhibits the same score as accuracy because the worst-case datasets only consist of one-way-missing links.

V. COMPARATIVE ANALYSES

In this section, we first experimentally compare semi-weighted and weighted graph representations to demonstrate the reason behind our choice of semi-weighted graph representations. Next, we set the proposed representation learning based on EDSoVS against the vector difference to showcase the performance of EDSoVS in our experimental results. Finally, we compare the spectral embedding approach with EDSoVS against four techniques involving neural networks,

TABLE X

PERFORMANCE SUMMARY OF ALL WORST-CASE DATASETS USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS AND THE EDSOVS. NOTE THAT PRECISION SCORE IS 100% AND RECALL EXHIBITS THE SAME SCORE AS ACCURACY BECAUSE THE DATASETS CONSIST OF ONLY ONE CLASS

Dataset	Accuracy	Precision	Recall	F1 Score
MITRE	96.88%	100.00%	96.88%	98.41%
Cisco G-1	96.09%	100.00%	96.09%	98.00%
Cisco G-7	98.77%	100.00%	98.77%	99.38%
Cisco G-8	98.58%	100.00%	98.58%	99.28%
Cisco G-11	99.36%	100.00%	99.36%	99.68%
Cisco G-12	98.02%	100.00%	98.02%	99.00%
Cisco G-13	90.38%	100.00%	90.38%	94.84%
Cisco G-14	85.03%	100.00%	85.03%	91.65%
Cisco G-16	96.05%	100.00%	96.05%	97.98%
Cisco G-17	94.54%	100.00%	94.54%	97.16%
Cisco G-18	99.01%	100.00%	99.01%	99.50%
Cisco G-19	84.48%	100.00%	84.48%	91.58%
Cisco G-20	71.55%	100.00%	71.55%	83.42%

namely, Node2Vec [13] and DeepWalk [16], and graph neural networks, namely, DGCN [17] and TADDY [18].

A. Comparison Between Semi-Weighted and Weighted Graph Representations

In Section IV-D, we presented our results using semi-weighted directed graphs. In this section, we introduce the performance of our model with weighted, directed graphs and compare them with semi-weighted, directed graphs. The results in Table XI justify the reason behind our choice of the semi-weighted graph representation. In the table, the semi-weighted graph representation consistently outperforms the weighted graph representation across all datasets. Although the difference in performance is often around one percentage point on many datasets, it is around seven points on Cisco G-13 and Cisco G-20. The overall performance can be attributed to only the expected links having associated weights, while anomalous links often carry zero weight. Furthermore, our approach carries out a binary classification task rather than a regression task of predicting link weights, hence, it does not depend on the weights as much. Overall, we found that the semi-weighted representation of directed graphs is a better approach for detecting anomalous links in all datasets.

B. Comparison Between EDSoVS and Difference Vector Representations

In Sections IV-D and V-A, we presented the empirical results using the External Direct Sum of Vector Spaces (EDSoVS) representation of *dyads*. This section presents the results for the difference vector representation of *dyads* and compares their performances. Table XII displays the performance comparison of the EDSOVS and difference vector representations of *dyads* on the semi-weighted directed graphs. EDSOVS vector representations outperform the difference vector representations by 2 to 15 points based on

TABLE XI

PERFORMANCE COMPARISON BETWEEN SEMI-WEIGHTED AND WEIGHTED REPRESENTATIONS OF THE DIRECTED GRAPHS USING EDSOVS ON ALL DATASETS

Dataset	Weight Variant	Accuracy	Precision	Recall	F1 Score
MITRE	Semi-Weighted	94.20%	91.30%	97.79%	94.41%
	Weighted	93.59%	91.91%	95.51%	93.67%
Cisco G-1	Semi-Weighted	94.67%	92.62%	97.20%	94.83%
	Weighted	93.37%	90.43%	96.97%	93.56%
Cisco G-7	Semi-Weighted	92.81%	89.08%	98.14%	93.28%
	Weighted	91.25%	89.34%	94.00%	91.52%
Cisco G-8	Semi-Weighted	90.07%	86.74%	99.04%	91.79%
	Weighted	89.40%	86.42%	97.85%	91.06%
Cisco G-11	Semi-Weighted	95.79%	93.53%	98.65%	95.97%
	Weighted	94.31%	92.61%	96.52%	94.46%
Cisco G-12	Semi-Weighted	86.93%	80.57%	98.67%	88.52%
	Weighted	85.74%	82.79%	92.46%	86.88%
Cisco G-13	Semi-Weighted	77.20%	74.72%	94.87%	82.26%
	Weighted	70.51%	67.18%	93.83%	77.11%
Cisco G-14	Semi-Weighted	86.12%	82.29%	92.72%	87.15%
	Weighted	82.50%	81.29%	84.16%	82.70%
Cisco G-16	Semi-Weighted	91.43%	87.57%	97.66%	92.14%
	Weighted	91.19%	88.14%	96.43%	91.83%
Cisco G-17	Semi-Weighted	79.32%	76.35%	97.29%	84.29%
	Weighted	78.68%	78.55%	92.88%	83.35%
Cisco G-18	Semi-Weighted	90.04%	86.14%	99.40%	91.68%
	Weighted	89.62%	86.30%	98.40%	91.26%
Cisco G-19	Semi-Weighted	86.63%	84.02%	90.58%	87.13%
	Weighted	85.64%	89.91%	81.20%	85.21%
Cisco G-20	Semi-Weighted	70.54%	68.38%	95.98%	78.22%
	Weighted	63.83%	62.51%	95.55%	77.51%

accuracy scores across different datasets. These findings experimentally support the detailed arguments and discussions presented in Section III-D. In summary, the EDSoVS maps the representations of two separate *dyads* to two different vectors by expanding the basis of their embedding vectors, while the difference vector representations may potentially map two separate *dyads* to the same vectors. Overall, we found that EDSoVS vector representation of *dyads* is a better approach for detecting anomalous links.

C. Comparison With Other Methods

In our comparative analyses, we compare our approach against two embedding techniques based on neural networks: Node2Vec [13] and DeepWalk [16], and two current techniques based on graph neural networks: representation learning for Dynamic graphs based on Graph Convolutional Networks (DGCN) [17] and Transformer-based Anomaly Detection framework for DYnamic graph (TADDY) [18]. Node2Vec and DeepWalk utilize random walk strategies to explore graph structures, and neural networks to generate node embeddings that effectively preserve the underlying topology. Deepwalk employs a truncated random walk, while Node2Vec introduces a biased random walk, enabling it to explore a graph more effectively. On the other hand, DGCN maximizes the

TABLE XII

PERFORMANCE COMPARISON BETWEEN EDSOVS AND DIFFERENCE VECTOR REPRESENTATIONS OF DYADS ON THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS ON ALL DATASETS

Dataset	Edge Representation	Accuracy	Precision	Recall	F1 Score
MITRE	EDSoVS	94.20%	91.30%	97.79%	94.41%
	Difference	92.82%	91.63%	94.27%	92.92%
Cisco G-1	EDSoVS	94.67%	92.62%	97.20%	94.83%
	Difference	91.81%	92.49%	91.10%	91.75%
Cisco G-7	EDSoVS	92.81%	89.08%	98.14%	93.28%
	Difference	83.53%	80.33%	89.58%	84.61%
Cisco G-8	EDSoVS	90.07%	86.74%	99.04%	91.79%
	Difference	88.74%	84.99%	90.30%	90.54%
Cisco G-11	EDSoVS	95.79%	93.53%	98.65%	95.97%
	Difference	94.28%	92.73%	96.30%	94.44%
Cisco G-12	EDSoVS	86.93%	80.57%	98.67%	88.52%
	Difference	85.53%	80.62%	94.80%	86.87%
Cisco G-13	EDSoVS	77.20%	74.72%	94.87%	82.26%
	Difference	75.88%	74.18%	92.37%	80.82%
Cisco G-14	EDSoVS	86.12%	82.29%	92.72%	87.15%
	Difference	71.77%	74.34%	66.94%	69.97%
Cisco G-16	EDSoVS	91.43%	87.57%	97.66%	92.14%
	Difference	88.45%	84.61%	94.18%	89.11%
Cisco G-17	EDSoVS	79.32%	76.35%	97.29%	84.29%
	Difference	74.69%	73.55%	90.76%	79.77%
Cisco G-18	EDSoVS	90.04%	86.14%	99.40%	91.68%
	Difference	88.72%	84.26%	98.74%	90.42%
Cisco G-19	EDSoVS	86.63%	84.02%	90.58%	87.13%
	Difference	85.30%	84.98%	85.71%	85.32%
Cisco G-20	EDSoVS	70.54%	68.38%	95.98%	78.22%
	Difference	61.70%	60.40%	83.92%	69.20%

mutual information (MI) between local node representations and corresponding global graph representations to effectively capture the global structure information of a graph snapshot. In addition, DGCN utilizes long short-term memory (LSTM) to update the weight parameters of GCN and capture temporal information. TADDY is a node encoding strategy that effectively represents the structural and temporal roles of each node in the dynamic graphs. It utilizes a dynamic graph transformer model to capture the spatial-temporal information within the graph. The time complexity of both Node2Vec and DeepWalk is $\mathcal{O}(|V|\log|V|)$ [40], where $|V|$ is the number of nodes in a graph snapshot. The time complexity of DGCN is $\mathcal{O}(|E|\bar{d}\log\bar{d})$ [17], and TADDY is $\mathcal{O}(\tau k|E|I + T|\bar{V}|^2)$ [18], where $|E|$ is the number of edges, $\bar{d}$ is the average degree of a node, $|\bar{V}|$ is the average number of nodes, τ is the size of the time window, k is the number of contextual nodes, I is the number of iterations, and T is the total number of graph snapshots.

We present the comparative evaluation of Node2Vec, DeepWalk, DGCN, and TADDY with our proposed model in Table XIII. In the table, the average accuracy scores of Node2Vec vary between 68.79% to 91.04%, DeepWalk vary between 68.19% to 90.41%, DGCN vary between 69.11% to 91.54%, and TADDY vary between 69.39% to 92.12%. Node2Vec and DeepWalk perform best on the Cisco G-16

TABLE XIII

PERFORMANCE COMPARISON OF OUR PROPOSED MODEL USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS AND EDSOVS WITH OTHER NEURAL NETWORKS AND GRAPH NEURAL NETWORKS BASED TECHNIQUES

Dataset	Method	Accuracy	Precision	Recall	F1 Score
MITRE	Spectral Embedding	94.20%	91.30%	97.79%	94.41%
	Node2Vec	88.59%	87.15%	90.81%	88.88%
	DeepWalk	86.98%	84.37%	95.05%	88.64%
	TADDY	90.31%	88.01%	90.44%	89.21%
	DGCN	89.76%	87.46%	89.89%	88.66%
Cisco G-1	Spectral Embedding	94.67%	92.62%	97.20%	94.83%
	Node2Vec	86.24%	83.96%	93.95%	87.84%
	DeepWalk	80.75%	80.26%	81.75%	80.96%
	TADDY	92.12%	90.82%	92.62%	91.71%
	DGCN	87.78%	86.48%	90.28%	88.37%
Cisco G-7	Spectral Embedding	92.81%	89.08%	98.14%	93.28%
	Node2Vec	84.62%	83.80%	86.05%	84.87%
	DeepWalk	84.05%	83.43%	85.14%	84.23%
	TADDY	86.80%	85.00%	88.60%	86.79%
	DGCN	86.67%	85.87%	87.47%	86.66%
Cisco G-8	Spectral Embedding	90.07%	86.74%	99.04%	91.79%
	Node2Vec	86.98%	86.27%	88.57%	87.30%
	DeepWalk	83.40%	79.00%	92.35%	84.93%
	TADDY	88.31%	86.51%	90.76%	88.88%
	DGCN	88.76%	86.46%	89.21%	88.33%
Cisco G-11	Spectral Embedding	95.79%	93.53%	98.65%	95.97%
	Node2Vec	69.94%	69.25%	71.68%	70.43%
	DeepWalk	68.24%	68.42%	67.87%	68.12%
	TADDY	81.23%	80.83%	82.13%	81.47%
	DGCN	91.54%	91.14%	92.44%	91.79%
Cisco G-12	Spectral Embedding	86.93%	80.57%	98.67%	88.52%
	Node2Vec	83.22%	80.26%	90.15%	84.52%
	DeepWalk	83.28%	79.57%	91.01%	84.64%
	TADDY	85.17%	80.34%	92.76%	86.99%
	DGCN	84.10%	80.17%	90.69%	85.92%
Cisco G-13	Spectral Embedding	77.20%	74.72%	94.87%	82.26%
	Node2Vec	74.04%	73.47%	75.76%	74.52%
	DeepWalk	72.38%	72.98%	87.29%	77.52%
	TADDY	74.34%	73.65%	79.24%	77.55%
	DGCN	74.33%	73.04%	79.23%	77.54%
Cisco G-14	Spectral Embedding	86.12%	82.29%	92.72%	87.15%
	Node2Vec	76.31%	75.22%	78.71%	76.89%
	DeepWalk	74.36%	73.69%	76.45%	74.98%
	TADDY	78.09%	77.69%	83.09%	80.30%
	DGCN	80.45%	80.05%	85.45%	82.66%
Cisco G-16	Spectral Embedding	91.43%	87.57%	97.66%	92.14%
	Node2Vec	91.04%	81.76%	90.19%	85.95%
	DeepWalk	90.41%	80.83%	91.12%	85.46%
	TADDY	91.14%	84.87%	91.42%	87.64%
	DGCN	91.29%	84.72%	91.27%	87.49%
Cisco G-17	Spectral Embedding	79.32%	76.35%	97.29%	84.29%
	Node2Vec	78.25%	75.90%	92.96%	83.08%
	DeepWalk	77.56%	75.18%	92.25%	82.39%
	TADDY	78.32%	76.22%	93.22%	83.56%
	DGCN	78.87%	76.17%	92.97%	83.31%
Cisco G-18	Spectral Embedding	90.04%	86.14%	99.40%	91.68%
	Node2Vec	86.90%	84.77%	94.42%	88.50%
	DeepWalk	87.09%	83.38%	96.41%	88.82%
	TADDY	89.85%	85.45%	96.45%	89.86%
	DGCN	87.34%	84.94%	96.94%	89.36%
Cisco G-19	Spectral Embedding	86.63%	84.02%	90.58%	87.13%
	Node2Vec	78.34%	78.71%	78.47%	78.37%
	DeepWalk	78.93%	79.53%	78.50%	78.80%
	TADDY	79.65%	79.95%	84.45%	81.07%
	DGCN	81.23%	79.53%	86.03%	82.65%
Cisco G-20	Spectral Embedding	70.54%	68.38%	95.98%	78.22%
	Node2Vec	68.79%	68.15%	70.77%	69.41%
	DeepWalk	68.19%	68.23%	67.86%	68.10%
	TADDY	69.39%	68.19%	79.69%	74.40%
	DGCN	69.11%	66.11%	79.41%	72.15%

dataset, while DGCN achieves the best performance on the Cisco G-11 dataset. The number of nodes and edges in the Cisco G-11 and G-16 datasets are significantly fewer than in

other datasets. Conversely, TADDY achieves the best performance on the Cisco G-1 dataset, which has the highest density among all the datasets. TADDY utilizes an average pooling operation on the node embeddings to obtain dyad embeddings, whereas other methods use the Hadamard operator. As a result, TADDY performs better when the number of nodes and edges in a dataset is significantly higher than in other datasets. Nonetheless, our proposed approach consistently outperforms all methods across all datasets. The results validate that the EDSOVS vector representations can generate distinguishable dyad representations, which significantly enhances the performance of the proposed model.

VI. CONCLUSION

In this study, we proposed an approach to detect anomalous communication links in dynamically evolving networked systems. We leveraged the spectral embeddings to encode graph nodes into a low-dimensional space and generated embedding vectors for edge representations. Specifically, we proposed using the External Direct Sum of Vector Spaces (EDSOVS) to enable edge/dyad representation learning from vector representations of nodes. We used the dyad embeddings as features feeding a Support Vector Machine (SVM) classifier to detect anomalous links. We evaluated our proposed approach on a synthetically generated dataset simulating a real-world communication network provided by the MITRE corporation and twelve real-world datasets provided by Cisco. The empirical results show that the model defined over the semi-weighted representation of directed graphs using the EDSOVS vector representation of *dyads* achieves the best performance. Although some daily snapshots of five Cisco network datasets contain data collection artifacts that adversely affect the empirical results, the average accuracy scores of our proposed model vary between 70.54% to 95.75%, while the average F1 scores vary between 78.22% to 95.97% across different datasets. If we exclude the affected daily snapshots from our experiments, the average accuracy scores vary between 86.12% to 97.93%, while the average F1 scores vary between 87.13% to 97.95% across different datasets. Finally, we compared our approach with two embedding techniques based on neural networks and two current techniques based on graph neural networks, and showed that our proposed model consistently achieves better results on all datasets.

Our approach can be integrated into IDSs to detect unexpected communication behaviors in systems where the IP addresses of the hosts are relatively consistent, even in the presence of a DHCP server leasing the IP addresses to the hosts. Therefore, the approach can also be utilized to mitigate the impact of internal attacks, including Smurf and other DDoS attacks, executed within systems.

APPENDIX

PERFORMANCE SUMMARY OF INDIVIDUAL CISCO NETWORKS DATASETS

See Tables [XIV–XXV](#).

TABLE XIV

PERFORMANCE SUMMARY OF CISCO G-1 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 64

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	92.41%	88.70%	97.22%	92.76%
Day3	96.55%	96.13%	97.02%	96.57%
Day4	95.03%	93.03%	97.35%	95.14%
Average	94.67%	92.62%	97.20%	94.83%

TABLE XV

PERFORMANCE SUMMARY OF CISCO G-7 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 8

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	95.86%	94.36%	97.56%	95.93%
Day3	87.87%	81.18%	98.57%	89.03%
Day4	94.70%	91.70%	98.30%	94.88%
Average	92.81%	89.08%	98.14%	93.28%

TABLE XVI

PERFORMANCE SUMMARY OF CISCO G-8 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 16

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	74.35%	66.26%	99.22%	79.46%
Day3	97.73%	96.66%	98.87%	97.75%
Day4	98.13%	97.29%	99.02%	98.15%
Average	90.07%	86.74%	99.04%	91.79%

TABLE XVII

PERFORMANCE SUMMARY OF CISCO G-11 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 32

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	92.22%	87.46%	98.56%	92.68%
Day3	98.70%	98.58%	98.83%	98.70%
Day4	96.45%	94.56%	98.56%	96.52%
Average	95.79%	93.53%	98.65%	95.97%

TABLE XVIII

PERFORMANCE SUMMARY OF CISCO G-12 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 16

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	78.99%	70.73%	98.90%	82.48%
Day3	91.61%	86.71%	98.29%	92.13%
Day4	90.18%	84.26%	98.82%	90.96%
Average	86.93%	80.57%	98.67%	88.52%

TABLE XIX

PERFORMANCE SUMMARY OF CISCO G-13 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 128

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	54.00%	52.10%	99.28%	68.34%
Day3	89.17%	86.79%	92.41%	89.51%
Day4	88.44%	85.28%	92.92%	88.94%
Average	77.20%	74.72%	94.87%	82.26%

TABLE XX

PERFORMANCE SUMMARY OF CISCO G-14 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 16

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	86.13%	84.72%	89.68%	87.13%
Day3	86.41%	81.23%	94.72%	87.46%
Day4	85.82%	80.91%	93.75%	86.86%
Average	86.12%	82.29%	92.72%	87.15%

TABLE XXI

PERFORMANCE SUMMARY OF CISCO G-16 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 8

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	83.92%	76.56%	97.66%	85.83%
Day3	95.61%	92.62%	99.12%	95.76%
Day4	94.77%	93.52%	96.19%	94.84%
Average	91.43%	87.57%	97.66%	92.14%

TABLE XXII

PERFORMANCE SUMMARY OF CISCO G-17 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 4

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	54.21%	52.21%	98.94%	68.35%
Day3	94.71%	91.88%	98.09%	94.88%
Day4	89.03%	84.96%	94.84%	89.63%
Average	79.32%	76.35%	97.29%	84.29%

TABLE XXIII

PERFORMANCE SUMMARY OF CISCO G-18 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 4

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	75.25%	66.96%	99.56%	80.07%
Day3	96.74%	94.62%	99.10%	96.81%
Day4	98.14%	96.83%	99.53%	98.17%
Average	90.04%	86.14%	99.40%	91.68%

TABLE XXIV

PERFORMANCE SUMMARY OF CISCO G-19 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 8

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	89.35%	87.50%	91.67%	89.53%
Day3	88.17%	86.36%	90.48%	88.37%
Day4	82.38%	78.18%	89.58%	83.50%
Average	86.63%	84.02%	90.58%	87.13%

TABLE XXV

PERFORMANCE SUMMARY OF CISCO G-20 DATASET USING THE SEMI-WEIGHTED REPRESENTATION OF DIRECTED GRAPHS WITH EDSOVS AND BEST EMBEDDING SIZE 4

Test Dataset	Accuracy	Precision	Recall	F1 Score
Day2	50.18%	50.08%	100.00%	66.74%
Day3	92.22%	92.58%	91.79%	92.18%
Day4	69.20%	62.48%	96.15%	75.74%
Average	70.54%	68.38%	95.98%	78.22%

REFERENCES

- [1] M. S. M. Gismalla et al., "Survey on device to device (D2D) communication for 5GB/6G networks: Concept, applications, challenges, and future directions," *IEEE Access*, vol. 10, pp. 30792–30821, 2022.
- [2] J. I. Z. Chen and L.-T. Yeh, "Data forwarding in wireless body area networks," *J. Electron. Informat.*, vol. 2, no. 2, pp. 80–87, Jun. 2020.
- [3] J. Wang et al., "IoT-praetor: Undesired behaviors detection for IoT devices," *IEEE Internet Things J.*, vol. 8, no. 2, pp. 927–940, Jan. 2021.
- [4] G. Xue, M. Zhong, J. Li, J. Chen, C. Zhai, and R. Kong, "Dynamic network embedding survey," *Neurocomputing*, vol. 472, pp. 212–223, Feb. 2022.
- [5] B. L. Bars and A. Kalogeratos, "A probabilistic framework to node-level anomaly detection in communication networks," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, Apr. 2019, pp. 2188–2196.
- [6] K. Ding, J. Li, R. Bhanushali, and H. Liu, "Deep anomaly detection on attributed networks," in *Proc. IEEE Int. Conf. Data Min. (ICDM)*, May 2019, pp. 594–602.
- [7] S. Tian et al., "SAD: Semi-supervised anomaly detection on dynamic graphs," 2023, *arXiv:2305.13573*.
- [8] N. A. Heard, D. J. Weston, K. Platanioti, and D. J. Hand, "Bayesian anomaly detection methods for social networks," *Ann. Appl. Statist.*, vol. 4, no. 2, pp. 645–662, Jun. 2010.
- [9] B. Luo, R. C. Wilson, and E. R. Hancock, "Spectral embedding of graphs," *Pattern Recognit.*, vol. 36, no. 10, pp. 2213–2230, Oct. 2003.
- [10] P. Van Mieghem, "Directed graphs and mysterious complex eigenvalues," Tech. Rep., 2018.
- [11] F. Chung, "Laplacians and the cheeger inequality for directed graphs," *Ann. Combinatorics*, vol. 9, no. 1, pp. 1–19, Apr. 2005.
- [12] Q. Zheng and D. B. Skillicorn, "Spectral embedding of directed networks," in *Proc. IEEE/ACM Int. Conf. Adv. Social Netw. Anal. Mining (ASONAM)*, Aug. 2015, pp. 432–439.
- [13] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 855–864.
- [14] W. H. Greub, *Linear Algebra*, vol. 23. Berlin, Germany: Springer, 2012.
- [15] O. Madani, S. A. Averineni, and S. Gandham, "A dataset of networks of computing hosts," in *Proc. ACM Int. Workshop Secur. Privacy Analytics*, Apr. 2022, pp. 100–104.
- [16] B. Perozzi, R. Al-Rfou, and S. Skiena, "DeepWalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2014, pp. 701–710.
- [17] C. Gao, J. Zhu, F. Zhang, Z. Wang, and X. Li, "A novel representation learning for dynamic graphs based on graph convolutional networks," *IEEE Trans. Cybern.*, vol. 53, no. 6, pp. 3599–3612, Jun. 2023.
- [18] Y. Liu et al., "Anomaly detection in dynamic graphs via transformer," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12081–12094, Dec. 2023.
- [19] M. S. Rahman, S. Halder, M. A. Uddin, and U. K. Acharjee, "An efficient hybrid system for anomaly detection in social networks," *Cybersecurity*, vol. 4, no. 1, pp. 1–11, Dec. 2021.
- [20] W. Gu, F. Gao, R. Li, and J. Zhang, "Learning universal network representation via link prediction by graph convolutional neural network," *J. Social Comput.*, vol. 2, no. 1, pp. 43–51, Mar. 2021.
- [21] R. Feng, Y. Yang, W. Hu, F. Wu, and Y. Zhang, "Representation learning for scale-free networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, vol. 32, no. 1, pp. 282–289.
- [22] T. Liu, A. Qi, Y. Hou, and X. Chang, "Method for network anomaly detection based on Bayesian statistical model with time slicing," in *Proc. 7th World Congr. Intell. Control Autom.*, Jun. 2008, pp. 3359–3362.
- [23] F. Xia et al., "CenGCN: Centralized convolutional networks with vertex imbalance for scale-free graphs," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4555–4569, May 2023.
- [24] L. Cai, "Structural temporal graph neural networks for anomaly detection in dynamic graphs," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage. (CIKM)*, Oct. 2021, pp. 3747–3756.
- [25] X. Xu, H. Wang, A. Lal, C. A. Gunter, and B. Li, "EDoG: Adversarial edge detection for graph neural networks," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Feb. 2023, pp. 291–305.
- [26] I. J. King and H. H. Huang, "Euler: Detecting network lateral movement via scalable temporal link prediction," *ACM Trans. Privacy Secur.*, vol. 26, no. 3, pp. 1–36, 2022.
- [27] W. W. Lo, S. Layeghy, M. Sarhan, M. Gallagher, and M. Portmann, "E-GraphSAGE: A graph neural network based intrusion detection system for IoT," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, Apr. 2022, pp. 1–9.
- [28] S. Bhatia, R. Liu, B. Hooi, M. Yoon, K. Shin, and C. Faloutsos, "Real-time anomaly detection in edge streams," *ACM Trans. Knowl. Discovery Data*, vol. 16, no. 4, pp. 1–22, Aug. 2022.
- [29] D. Eswaran and C. Faloutsos, "SedanSpot: Detecting anomalies in edge streams," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2018, pp. 953–958.
- [30] X. Zhang, Y. He, N. Brugnone, M. Perlmutter, and M. Hirn, "MagNet: A neural network for directed graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 27003–27015.
- [31] Z. Tong, Y. Liang, C. Sun, D. S. Rosenblum, and A. Lim, "Directed graph convolutional network," 2020, *arXiv:2004.13970*.
- [32] J. Gasteiger, J. Groß, and S. Günnemann, "Directional message passing for molecular graphs," 2020, *arXiv:2003.03123*.
- [33] M. Belkin and P. Niyogi, "Laplacian eigenmaps and spectral techniques for embedding and clustering," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 14, 2001.
- [34] S. P. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
- [35] J. Leskovec, D. Chakrabarti, J. Kleinberg, C. Faloutsos, and Z. Ghahramani, "Kronecker graphs: An approach to modeling networks," *J. Mach. Learn. Res.*, vol. 11, no. 2, pp. 985–1042, 2010.
- [36] L. Xu, M. Skoularidou, A. Cuesta-Infante, and K. Veeramachaneni, "Modeling tabular data using conditional GAN," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019.
- [37] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2016, pp. 785–794.
- [38] V. Y. Pan and Z. Q. Chen, "The complexity of the matrix eigenproblem," in *Proc. 31st Annu. ACM Symp. Theory Comput.*, May 1999, pp. 507–516.
- [39] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, Sep. 1995.
- [40] T. Pimentel, A. Veloso, and N. Ziviani, "Fast node embeddings: Learning ego-centric representations," in *Proc. ICLR*, 2018, pp. 1–22.
- [41] M. Hassan, "Detecting anomalous links in dynamically evolving complex systems," Ph.D. dissertation, Univ. Louisiana Lafayette, Lafayette, LA, USA, 2024.